

递归学习笔记

递归是一种把大问题分解成同构小问题的思考方式写递归函数时最重要的不是立刻追踪每一层调用而是先回答两个问题分解规则是什么基线条件是什么只要这两件事想清楚了剩下的就交给调用栈。

很多人第一次读递归代码时会忍不住在脑子里展开整个调用过程展开三层之后就会迷失更有效的方法是相信递归假设假设函数对更小规模的输入已经正确只验证缩小规模之后正确使用其结果这一步这就是归纳法在程序里的样子。

分解与基线

递归函数的第一行几乎总是基线条件基线条件回答输入小到什么程度时答案可以直接给出例如阶乘函数里为0或1时直接返回1目录遍历里，当前路径不是目录时直接处理文件本身。

分解规则则回答如何把规模为n的问题拆成规模更小的同构子问题关键约束是每次分解都必须向基线条件靠近否则调用栈会一直加深最终以栈溢出收场。

写完这两个部分之后还应当检查终止性是否存在一条输入路径使分解永远不触达基线对负数输入求阶乘就是经典反例因此健壮的实现会先把非法输入挡在门外。

调用栈与栈帧

每一次递归调用都会在调用栈上压入一个新的栈帧保存参数局部变量和返回地址递归深度就是同时存在的栈帧数量理解这一点之后递归太深会爆栈就不再神秘栈空间是有限的深度超过上限就会抛出错误。

把递归改写成迭代本质上是自己维护一个显式的栈把系统隐式提供的服务变成可见的数据结构改写之后空间占用未必更小但深度上限从语言栈限制变成了堆内存限制通常宽裕得多。

递归与归纳

数学归纳法与递归互为镜像归纳法从小结论走向大结论递归从大问题调用小解法证明归纳时我们验证基例与归纳步写递归时我们验证基线条件与分解规则两者的可靠性来自同一个源头因此递归不是玄学而是可以逐步验证的工程结构。

练习建议先写三遍阶乘再写两遍目录遍历最后写一次汉诺塔三者的难度递增但检查点完全一致写完之后试着向别人解释一遍解释不通的地方就是理解没有闭合的地方。

尾递归的启示

如果一个递归调用是函数体的最后一个动作且返回值不再参与后续计算就称其为尾递归尾递归理论上可以被优化成循环因为旧栈帧不再被需要可惜主流引擎并没有实现尾调用优化这个概念更多是理解哪些递归容易被改写成循环的思维工具。

递归不是炫技它是处理树形结构分治算法和回溯搜索时最自然的表达下一份笔记里我们会用递归遍历一棵真正的树并把每一次调用都画在纸上。